資訊工程導論
第五章:演算法(Algorithm)

本講義未經同意請勿自行翻印

教科書: Computer Science an overview, 13th Edition, J. Glenn Brookshear and D. Brylow, Pearson, Addison Wesley, Pub. (滄海圖書代理) 中英文版

第 5 章 演算法

- 5.1 演算法的概念
- 5.2 演算法的表示法
- 5.3 演算法的探索
- 5.4 迭代結構
- 5.5 遞迴結構
- 5.6 效率與正確性

© 滄海圖書

課程目標(Lesson Objective)

本課結束後，你將能夠：

- 了解演算法的正式定義與意義
- 了解在撰寫程式之前或進行程式設計期間，用以表達演算法的過程（演算法表示法）
- 了解問題解決的理念，並將這些理念與演算法的發現(discovery)與表示(representation)相聯繫
- 了解兩種演算法結構：迭代(iterative)與遞迴(recursive)
- 了解演算法的效率(efficiency)與正確性(correctness)

© 滄海圖書



To limber up (先暖身)

- Video Clips
  - **CES最新黑科技! AI桌面寵物誕生 還有會翻譯的智慧眼鏡 超強大黃蜂按摩椅**

    https://www.youtube.com/watch?v=sMyOp_FUbCY
  - **十三分鐘略懂 AI 技術：機器學習、深度學習技術原理及延伸應用**

    https://www.youtube.com/watch?v=UGdG4WpluJ8&list=PLIVxtdWEhk9rEJn9Ph926MEe0gagaR_G0&index=6
- What is an algorithm(演算法)
  - Before you can answer this question, watch the following Youtube vlog.
    - https://www.youtube.com/watch?v=6hfOvs8pY1k

© 滄海圖書

5.1 演算法的概念

- 演算法
  - 數值表示法的轉換演算法
  - 資料錯誤偵測與更正的演算法
  - 資料壓縮與解壓縮的演算法
- 很多研究學者相信人類心智的各項活動就是一個演算法

© 滄海圖書

演算法的正式定義

- 演算法由一組有順序性(sequencing)且清楚(unambiguous)可執行(executable)的步驟所構成的集合，其程序最終可結果
- 演算法的步驟有不同執行方式
  - 線性 (1, 2, 3, …)
  - 平行 (多處理器)
  - 前因後果 (電路)

© 滄海圖書

演算法的正式定義

- 可終結的程序
  - 最終產生出一個結果
  - 可包含無止境的程序
    - 醫院的監控設備
    - 長除法演算法
- 不可終結的程序
  - 不會產生出結果

*Computer scientists use the term effective(效率) to capture the concept of being executable.

© 滄海圖書

演算法的抽象本質

- 演算法與它的表示法之間具有差異
  - 類比：如故事與書本之間的差異
- 程式是演算法的表示方式
- 處理程序為演算法的執行運作

© 滄海圖書

Question(s)

- Summarize the distinctions between a process, an algorithm, and a program.

© 滄海圖書

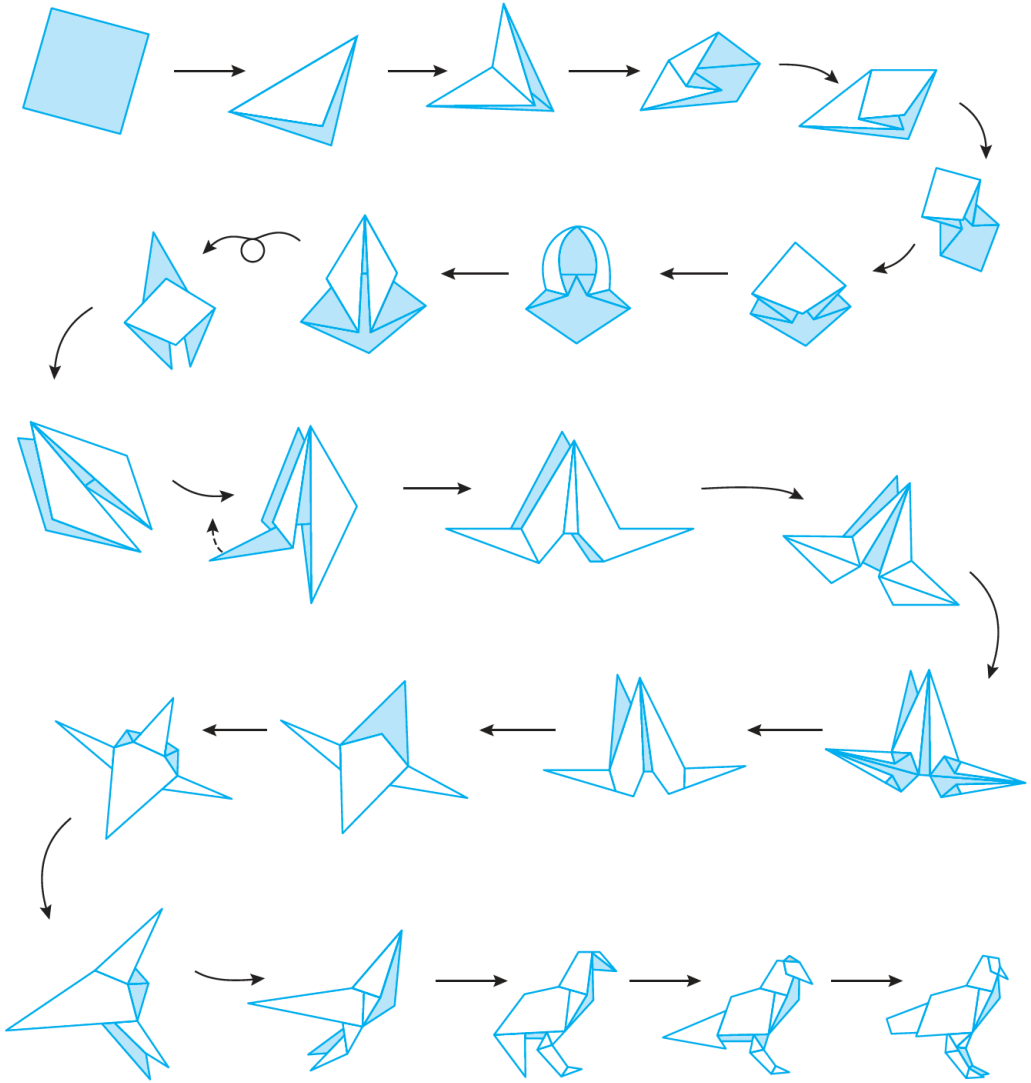


5.2 演算法的表示法

- 可由定義完善的原式(primitive)來表示
  - 一群原式可組成一個程式語言
  - Primitive=Syntax+Semantics
- 可由虛擬碼(pseudocode)來表示
  - 虛擬碼介於自然語言(natural language)與程式語言(programming language)

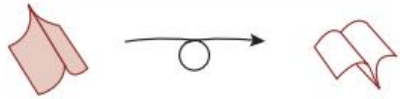
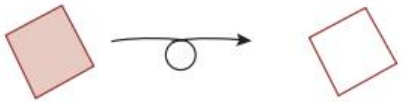
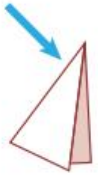
© 滄海圖書

圖5.2 以一張色紙摺出一隻鳥



© 滄海圖書

Origami primitives(原式) (圖5.3)

Syntax	Semantics
	Turn paper over as in 
Shade one side of paper	Distinguishes between different sides of paper as in 
	Represents a valley fold so that  represents 
	Represents a mountain fold so that  represents 
	Fold over so that  produces 
	Push in so that  produces 

© 滄海圖書

設計虛擬碼語言

- 選擇一個常見的程式語言
- 放寬程式語言的部分規則
- 加入一些自然語言
- 使用一致且精準的符號
- 我們使用類似 Python 的虛擬碼

*Natural language and pseudocode describe algorithms so that humans can understand them.

© 滄海圖書

Example: From Pseudocode to Python

procedure *max*(a_1, a_2, …., a_n: integers)

max := a_1

for *i* := 2 to *n*

if *max* < a_i then *max* := a_i

return *max*{*max* is the largest element}

~(Rosen, 2013)

```python
def max(list):
    m_val = list[0]  # 初始化最大值為列表的第一個元素
    for i in range(1, len(list)):  # 從第二個元素開始迭代
        if m_val < list[i]:  # 如果當前元素大於目前的最大值
            m_val = list[i]  # 更新最大值
    return m_val  # 返回最大值

# main
my_list = [2, 8, 3, 6, 10]  # 定義一個數字列表
max_value = max(my_list)  # 呼叫 max 函數並儲存返回值
print(max_value)  # 輸出最大值
```

© 滄海圖書

虛擬碼原式(primitive)

- 指定陳述(Assignment)

  name = expression

- 陳述

  RemainingFunds = CheckingBalance + SavingsBalance

© 滄海圖書

虛擬碼原式 (續)

- 條件式選擇(Conditional selection)

```
if (condition):
        activity
else:
        activity
```

- 範例

```
if (year is leap year):
        daily total = total / 366
else:
        daily total = total / 365
```


© 滄海圖書

虛擬碼原式 (續)

- 範例

```
if (condition):
    activity
```

```
If (銷售額下降):
    售價調降5%
```


© 滄海圖書

虛擬碼原式 (續)

- 縮排表示巢狀(nested)條件

```
If (沒下雨):
    if(氣溫 == 高溫):
        去游泳
    else:
        打高爾夫球
else:
    看電視
```

*Indentation(縮排) often enhances the readability of a program.

© 滄海圖書

虛擬碼原式 (續)

- 反覆執行(Repeated execution)

```
while (condition):
        body
```

- Example

```
    while (tickets remain to be sold):
        sell a ticket
```

© 滄海圖書



虛擬碼原式 (續)

- 定義函數

```
def name():
```

- 範例

```
def ProcessLoan():
```

- 執行函數

```
if (...):
    ProcessLoan()
else:
    RejectApplication()
```


© 滄海圖書

圖5.4 Python 中的函數 Greetings

```
def Greetings():
    Count = 3
    while (Count > 0):
        print the message "Hello"
        Count = Count - 1
```


© 滄海圖書

虛擬碼原式 (續)

- 使用參數(parameter)

```
def Sort(List):
```

- 以不同串列來執行排序

```
Sort(the organization's membership list)

Sort(the wedding guest list)
```


© 滄海圖書

Naming Convention

- Pascal casing
  - EstimatedArrivalTime
- Camel casing
  - estimated ArrivalTime
- Other
  - estimated_arrival_time

© 滄海圖書



Question?

- What are the building blocks of algorithms?
- What is “indentation”?

© 滄海圖書



Some Important Points

- 順序(sequencing)、選擇(selection)與反覆(iteration)是演算法的基本積木。
- 任何演算法都可以僅用順序、選擇與反覆來構造。
- 反覆（迭代; Iteration）是將演算法的一部分重複(repetition)執行，直到條件(condition)成立，或重複指定次數。
- 演算法可以彼此組合以形成新的演算法。

© 滄海圖書



休息片刻











Tilly

© 滄海圖書

5.3 演算法的探索(Discovery)

- 開發程式的第一步
- 比較是藝術，而非是一種技能
- 是具有挑戰性的任務

*Thus, to understand how algorithms are discovered is to understand the problem-solving process

*建立計算方法需要一個反覆不斷探勘的過程，才能將想法轉化成具體可行的形式。

© 滄海圖書



Polya(1945) 的解題(Problem Solving)階段論

1. 瞭解問題
2. 制訂解決問題的方法
3. 執行該方法
4. 評估方法的準確性以及該方法用來解決其他問題的可能性

- 1. Understand the problem.
- 2. Devise a plan for solving the problem.
- 3. Carry out the plan.
- 4. Evaluate the solution for accuracy and its potential as a tool for solving other problems.

© 滄海圖書

Polya 的階段論應用於程式開發上

1. 瞭解問題
2. 找出一個可能解決該問題的演算法
3. 制定出該演算法的流程並以程式表達
4. 評估程式的準確性以及該程式用來解決其他問題的可能性

© 滄海圖書

Two Versions In Comparison

Phase 1. Understand the problem.
Phase 2. Devise a plan for solving the problem.
Phase 3. Carry out the plan.
Phase 4. Evaluate the solution for accuracy and for its potential as a tool for solving other problems.

Phase 1. Understand the problem.
Phase 2. Get an idea of how an algorithmic function might solve the problem.
Phase 3. Formulate the algorithm and represent it as a program.
Phase 4. Evaluate the program for accuracy and for its potential as a tool for solving other problems.

© 滄海圖書



邁出第一步(Getting a Foot in the Door)

- 反推法
- 解決相關但較簡易的問題
  - 放寬問題的某些限制
  - 先解決某些特定的問題 (由下而上(bottom-up)的方法論)
- 逐步求精法(stepwise refinement)：把一個問題分解為許多小問題 (由上而下(top-down)的方法論)
- Top-down and bottom-up methods often complement each other; e.g. Jigsaw puzzles
- Usually,
  - planning->top-down
  - implementation -> bottom-up



~google images

小孩的年齡問題(Case Study 1)

- A被要求算出B的三個小孩年齡
  - B告訴A三個小孩年齡的乘積為36
  - A還需要一個提示
  - B告訴A三個小孩的年齡總和
  - A還是認為需要再一個提示
  - B告訴A年紀最大的小孩會彈鋼琴
  - A知道了三個小孩的年齡
- 這三個小孩的年齡為何？

- Person A is charged with the task of determining the ages of B's three children.
  - B tells A that the product of the children's ages is 36.
  - A replies that another clue is required.
  - B tells A the sum of the children's ages.
  - A replies that another clue is needed.
  - B tells A that the oldest child plays the piano.
  - A tells B the ages of the three children.
- How old are the three children?

© 滄海圖書

圖5.5 分析可能性

a. 三數的乘積為 36

(1,1,36)	(1,6,6)
(1,2,18)	(2,2,9)
(1,3,12)	(2,3,6)
(1,4,9)	(3,3,4)

b. (a) 部分的三數總和

1 + 1 + 36 = 38	1 + 6 + 6 = 13
1 + 2 + 18 = 21	2 + 2 + 9 = 13
1 + 3 + 12 = 16	2 + 3 + 6 = 11
1 + 4 + 9 = 14	3 + 3 + 4 = 10

© 滄海圖書

Order of Finishing a Race(Case Study 2)

Before A, B, C, and D ran a race they made the following predictions:

1. A predicted that B would win.
2. B predicted that D would be last
3. C predicted that A would be third.
4. D predicted that A's prediction would be correct.

Only one of these predictions was true, and this was the prediction made by the winner. In what order did A, B, C, and D finish the race?

© 滄海圖書



Question Thinking?

- Is it possible to fully understand the question in the first place?(p.273~274)

See Case Study 1

© 滄海圖書



To Wrap Up

- Generally, program development is not the process of solving a particular instance of a problem but rather of finding a general algorithm that can be used to solve all instances of the problem.

一般來說，程式開發不是解決問題的特定實例的過程，而是尋找可用於解決問題的所有實例的通用演算法的過程。

© 滄海圖書



In-Class Exercise

給定一個正整數) n(positive integer n，找出一組正整數數列，其總和(sum)為*n*，且所有數的乘積(product)是最大的。

例如若*n* = 4, 則該數列應為2, 2，因為2 × 2 比1 × 1 × 1 × 1, 2 × 1 × 1 和3 × 1 都大。若*n* = 5，則該數列(list)為2, 3。

a.試說明你如何邁出第一步？

b. 將此問題的algorithm寫成其pseudocode?

c.若*n* = 2001，試問該數列為何？

© 滄海圖書

Problem-Solving Steps

1={(1)}

2= {(1, 1);(2)}

3 ={(1 1 1); (1 2); **(3)**}
4 = {(1 1 1 1); (1 1 2); (1 3); **(2 2)}**
5 = {…; **(3 2)**}

6 ={(1 1 1 1 1 1); (1 1 2 1 1); (1 1 3 1); (1 2 3); **(3 3)}**

7 = {(1 2 3 1); (4 3); (**3 2 2**)}

8 = {…; **(3 3 2)}**

9 = {…; (3 3 3)}

- 3/3= 1…0=> 3
- 4/3= 1…1=> 2 2
- 5/3= 1…2= 3 2
- 6/3 = 2..0 = 33
- 7/3 = 2..1 = 3 22
- 8/3= 2…2 => 33 2
- 9/3 = 3…0=>333

© 滄海圖書



Algorithm to Python

function integer **largest_Product_Set**(n: inter)

if (n == 1 or n == 2):

the answer is the list containing the single value n

else:

Divide n by 3, obtaining a quotient q and a remainder r

if (r == 0):

the answer is the list containing q 3s

if (r == 1):

the answer is the list containing (q - 1) 3s and two 2s

if (r == 2):

the answer is the list containing q 3s and one 2

```python
def largest_Product_Set(n):
    if n == 1 or n == 2:
        return [n]  # 如果 n 是 1 或 2，返回 n 自身作為列表
    else:
        q, r = divmod(n, 3)  # 將 n 除以 3，得到商和餘數
        if r == 0:
            return [3] * q  # 如果餘數為 0，返回 q 個 3
        elif r == 1:
            return [3] * (q - 1) + [2, 2]  # 如果餘數為 1，返回 (q-1) 個 3 和兩個 2
        elif r == 2:
            return [3] * q + [2]  # 如果餘數為 2，返回 q 個 3 和一個 2
```

```python
result = largest_Product_Set(2001)
print(result)
print(len(result))
```

To Wrap Up:演算法的特性

- **輸入(Input)**
  - 演算法由一個特定集合中得到輸入值。
- **輸出(Output)**
  - 演算法中每有一個輸入值都將自某特定集合中得到輸出值。輸出值則為程式的結果。
- **確定性(Definiteness)**
  - 演算法的步驟定義必須非常明確。
- **有限性(Finiteness)**
  - 任何輸入值在演算法中必須在有限（也許是非常大的）步驟後便能得到輸出。
- **有效性(Effectiveness)**
  - 演算法的步驟必須能夠在有限的時間內確實執行。
- **通用性(generality)**
  - 演算法的過程必須能適用於所有希望解決之問題，而非某些特殊的輸入

Algorithmic Structures(演算法結構)

迭代結構（Iterative Structures）

- 以循環方式重複執行一組指令。

遞迴結構（Recursive Structures）

- 在實作重複性活動時，提供對迴圈範式的替代方案。
- 迴圈的作法是：完成一整組指令後再重複該組指令。
- 遞迴則是將這組指令當作其自身的子任務來重複執行。

© 滄海圖書



Algorithmic Structures(演算法結構)

- Iteration(迭代)

```
for i in range(10):
    i += 1
```

base case(基本情況)/degenerative case(衰退情況)/ base step(基礎步驟)

```
def increase_num_recursive(i, n):
    if i < n:
        print(i)
        increase_num_recursive(i + 1, n)
```

- Recursion(遞迴)

```
def fib(n):
    if n == 0 or n == 1:
        return 1
    else:
        return fib( n-1 ) + fib( n-2 )
```

```
def fib(n):
    if n == 0 or n == 1:
        return 1
    else:
        fib_0 = 1
        fib_1 = 1
        for i in range(2, n+1 ):
            fib_n = fib_0 + fib_1
            fib_0 = fib_1
            fib_1 = fib_n
        return fib_n
```



費氏數(Fibonacci number)

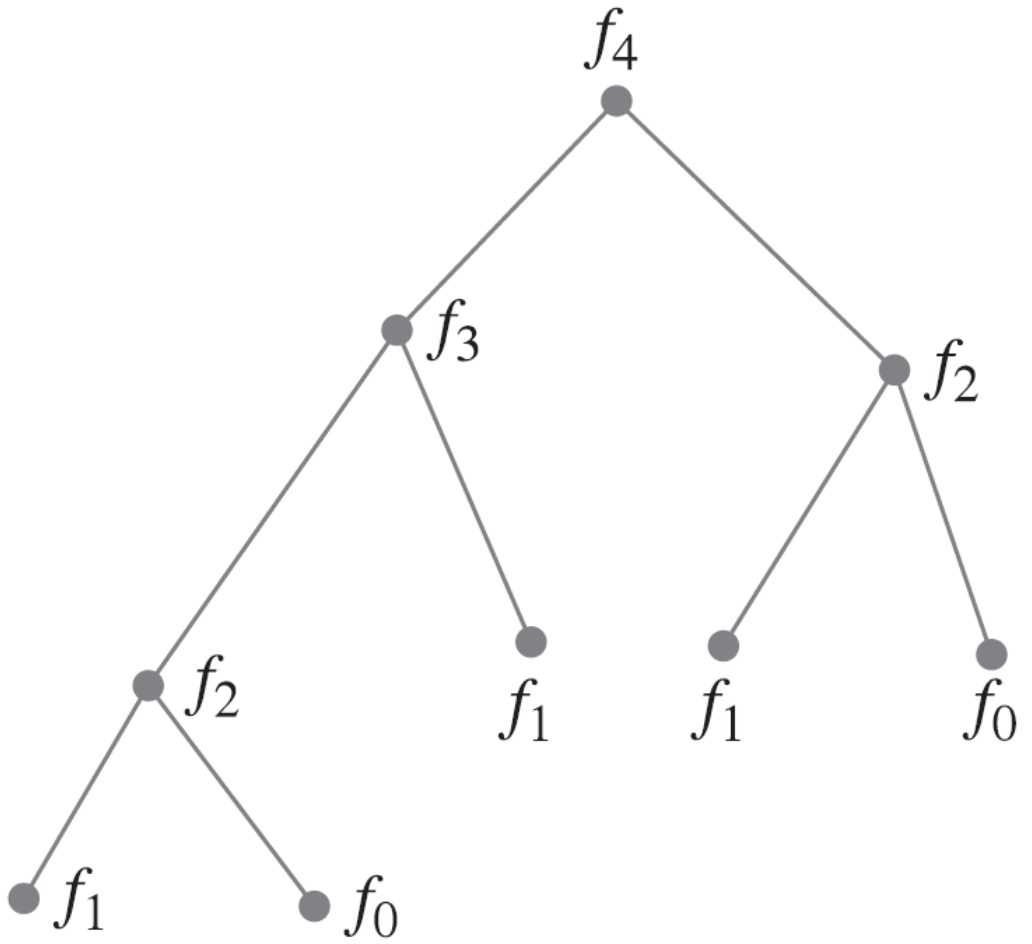
- Definition

費氏數(Fibonacci number)

f_0、f_1、f_2、…，是以等式$f_0 = 0$、$f_1 = 1$，以及

當$n = 2, 3, \ldots$，$f_n = f_{n-1} + f_{n-2}$來定義。

~(Rosen, 2013)



*藉由把問題簡化到更小輸入的相同問題，來解決原問題的演算法稱為遞迴演算法

© 滄海圖書

What about n!?

```
1. def factorial(n):
2.     result = 1
3.     for i in range(1, n+1):
4.         result [redacted]
5.     return result
```

```
1. def factorial(n):
2.     if n == 0 or n == 1:
3.         return 1
4.     else:
5.         return [redacted]
```

© 滄海圖書



遞迴(Recursion) v.s. 迭代(Iteration)

相同點

- 皆為重複執行一組指令
- 兩者都依賴條件來判斷何時停止

不同點

- 遞迴本質上是一次（再）呼叫方法/函式
- 迭代是以迴圈重複執行，直到滿足某個條件
- 遞迴是一種處理方式，總是套用在函式上

© 滄海圖書



Key Differences between Recursion and Iteration

- 遞迴的終止由條件判斷式決定（基本情況 / 退化情況 / 基底步驟）。
- 迭代敘述的終止由控制變數的值決定。
- 無限遞迴可能導致系統當機。
- 遞迴會重複呼叫方法，帶來在處理器時間與記憶體空間上的昂貴開銷；而迭代沒有這種呼叫開銷。
- 遞迴讓程式碼較短，而迭代讓程式碼較長。

5.4 迭代結構(Iterative Structure)

- 會反覆執行的一系列指令
- 例如：
  - 循序搜尋演算法(sequential search algorithm)
  - 插入排序演算法(insertion sort algorithm)

© 滄海圖書

圖5.6 循序搜尋法的虛擬碼

```
def Search(List, TargetValue):
  if (List is empty):
    Declare search a failure.
  else:
    Select the first entry in List to be TestEntry.
    while (TargetValue > TestEntry and
        there remain entries to be considered):
      Select the next entry in List as TestEntry.
    if (TargetValue = TestEntry):
      Declare search a success.
    else:
      Declare search a failure.
```

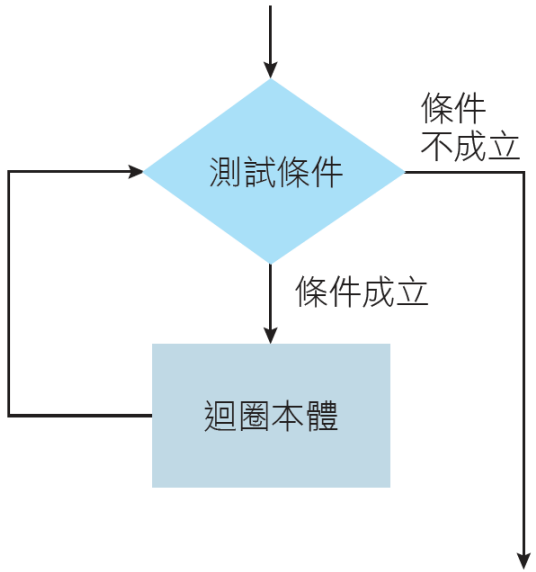
```python
def search(lst, target_value):
    if not lst:
        return "Search failure"
    else:
        for test_entry in lst:
            if target_value == test_entry:
                return "Search success"
    return "Search failure"
```


© 滄海圖書

迭代結構

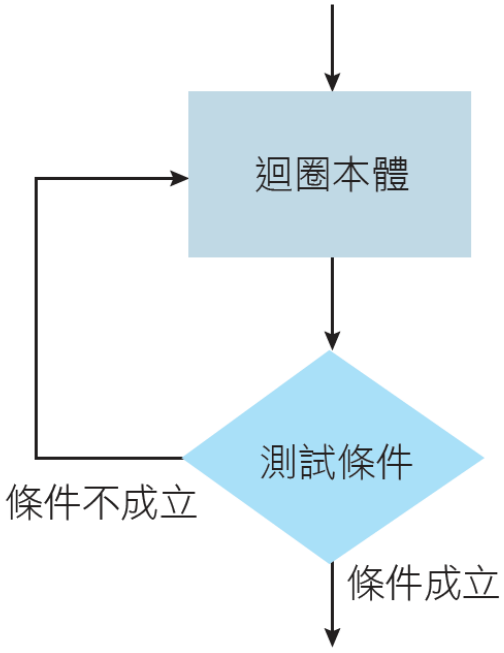
- 前測試迴圈(Pretest loop):

```
while (condition):
    body
```



- 後測試迴圈(Posttest loop):

```
repeat:
    activity
    until (condition)
```



Examples in Python

- Pretest loop

```
i = 1
while i < 6:
    print(i)
    i += 1
```

- Posttest loop

```
i = 1
while True:
    print(i)
    i += 1
    if i >= 6:
        break
```

```
# c version

int i = 1;
do{
   printf("%d\n", i);
   i = i + 1;

} while(i <= 6);
```

Note that in python, it does not support ‘do-while’ loop.

© 滄海圖書

Take a short breath! Try them out!

```
n = 1
while( n != 6 ):
    n = n + 2
    print(n)
```



- What would happen?

© 滄海圖書

Figure 5.7 Components of repetitive control(迴圈控制的組成要素)

Initialize: Establish an initial state that will be modified toward the termination condition

Test: Compare the current state to the termination condition and terminate the repetition if equal

Modify: Change the state in such a way that it moves toward the termination condition

初始化： 建立初始狀態，並且會趨向終止條件而變動。

測試： 比較目前狀態值與終止條件，若兩者相同，則終止反覆執行。

變動： 讓狀態值趨向終止條件改變。

圖5.11 插入排序(Insertion Sort)演算法的虛擬碼

```
def Sort(List):

N = 2

while(當 N 不超過名單長度):

    選擇名單的第一項做為軸心

    將軸心移到暫時位置,使名單中留下一個空格

    while(當空格之上有名字 and 名字大於軸心):

        將該名字移到空格中,在該名字上方留下一個空格

        將軸心移到名單中的空格中

    N = N + 1
```

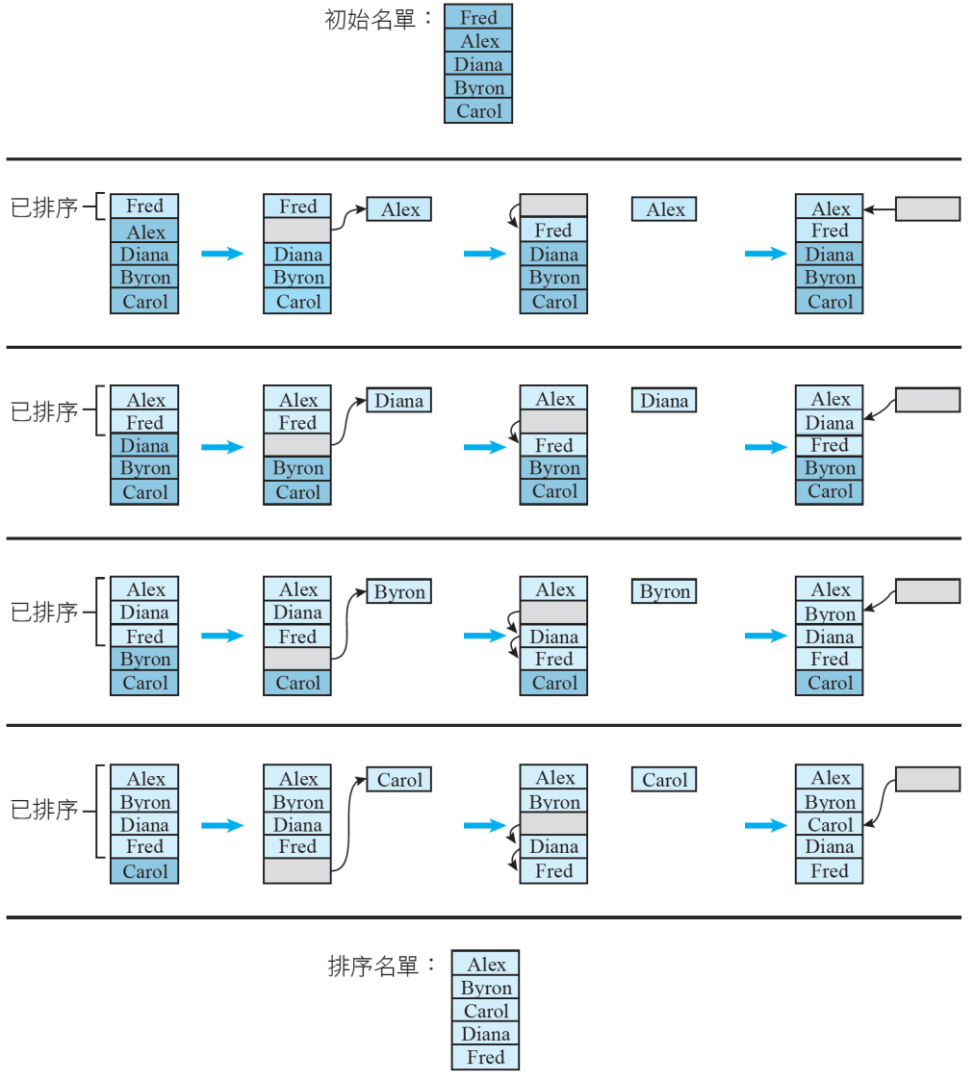
```
Run      main      test
C:\Users\PTsai\Pych
[1, 2, 3, 4, 5, 6]
```

```
44  def insertion_sort(lst):
45      n = 1
46      while n < len(lst):
47          pivot = lst[n]
48          j = n -1
49          while j >=0 and pivot < lst[j]:
50              lst[ j + 1] = lst[j]
51              j -= 1
52              lst[ j + 1 ] = pivot
53
54          n = n + 1
55
56      return lst

64  list_1 = [6, 5, 4, 3, 2, 1]
65  print(insertion_sort(list_1))
```


© 滄海圖書

圖5.10 以字母順序排序名單Fred, Alex, Diana, Byron 和Carol



© 滄海圖書

In-Class Exercise

- **Selection Sort(選擇排序法)**
  - A variation of the insertion sort algorithm
  - Q: Can you apply what we have learnt so far to selection sort algorithm?

```
def selection_sort (List):
    n = 1
    while (n is less than the length of List):
        j =n+ 1
        while ( j is not greater than length of List):
            if (the entry in position j is less than the entry in position n):
                interchange the two entrie
            j = j + 1
        n = n + 1
```

© 滄海圖書



```
def selection_sort (List):
    n = 1
    while (n is less than the length of List):
        j =n+ 1
        while ( j is not greater than length of List):
            if (the entry in position j is less than the entry in position n):
                interchange the two entrie
            j = j + 1
        n = n + 1
```

```
# Press the green button in the gutter to run the script.

lst_1 = [4, 3, 2, 1]
lst_2 = [1, 2, 3, 4]
lst_3 = [1, 3, 2, 4]

# print(insertion_sort(lst))
print(selection_sort(lst_3))
```



```
def selection_sort(lst):
    n = 0
    while n < len(lst):
        j = n + 1
        while j < len(lst):
            if lst[j] < lst[n]:
                temp = lst[j]
                lst[j] = lst[n]
                lst[n] = temp
            j = j + 1
        n = n + 1
    return lst
```



5.5 遞迴結構(Recursive Structure)

- 將自身當作副任務反覆執行

The execution of a procedure leads to another execution of the procedure.

- 過程中會產生多個程序的執行序，但除了一個之外，其他程序都要等待該程序執行完成

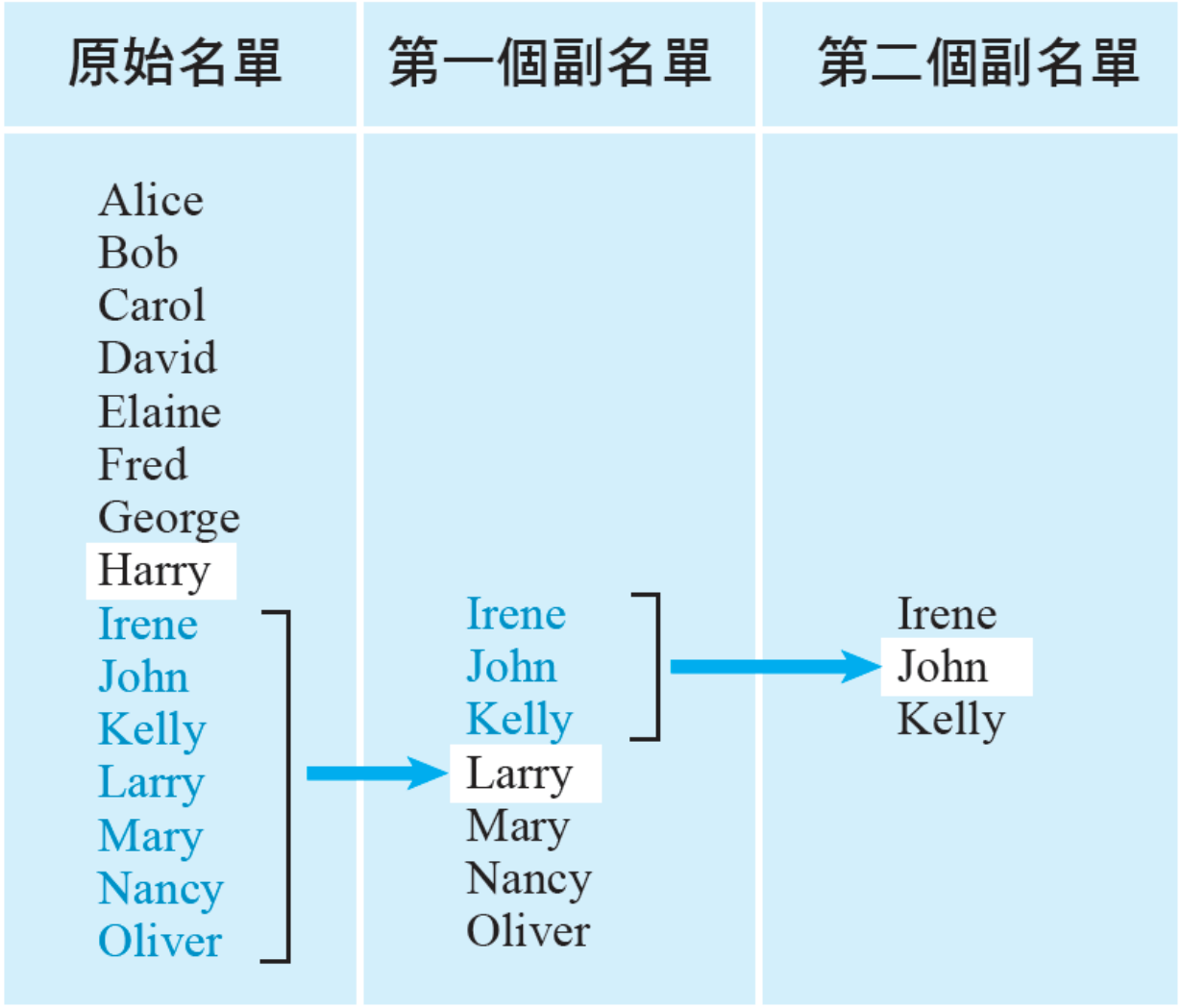
Multiple activations of the procedure are formed, all but one of which are waiting for other activations to complete

- 範例：二元搜尋法(binary search)



© 滄海圖書

圖5.12 應用本節所介紹的策略來搜尋名單中是否有John



© 滄海圖書

圖5.13 二元搜尋法的虛擬碼初版

```
if (List is empty):
  Report that the search failed.
else:
  TestEntry = the "middle" entry in the List
  if (TargetValue == TestEntry):
    Report that the search succeeded.
  if (TargetValue < TestEntry):
    Search() the portion of List preceding TestEntry for
      TargetValue, and report the result of that search.
  if (TargetValue > TestEntry):
    Search() the portion of List following TestEntry for
      TargetValue, and report the result of that search.
```


© 滄海圖書

Example: Binary Search in Python (Iteration)

```python
def binary_search_iterative(list_lst, target_value):
    low = 0
    high = len(list_lst) - 1
    while low <= high:
        mid = (low + high) // 2
        test_entry = list_lst[mid]

        if target_value == test_entry:
            print(f"Search succeeded, found target value: {target_value}")
            return mid  # Return the index where the target is found

        elif target_value < test_entry:
            high = mid - 1  # Search in the left half

        else:
            low = mid + 1  # Search in the right half

    print(f"Cannot find the desired target value: {target_value}")
    return -1  # Target not found

list = [1, 2, 3, 4, 5, 6 ]
binary_search_iterative(list, target_value: 6)
```

```
C:\Users\PTsai\PycharmProjects\pythonPro
Search succeeded, found target value: 6
```

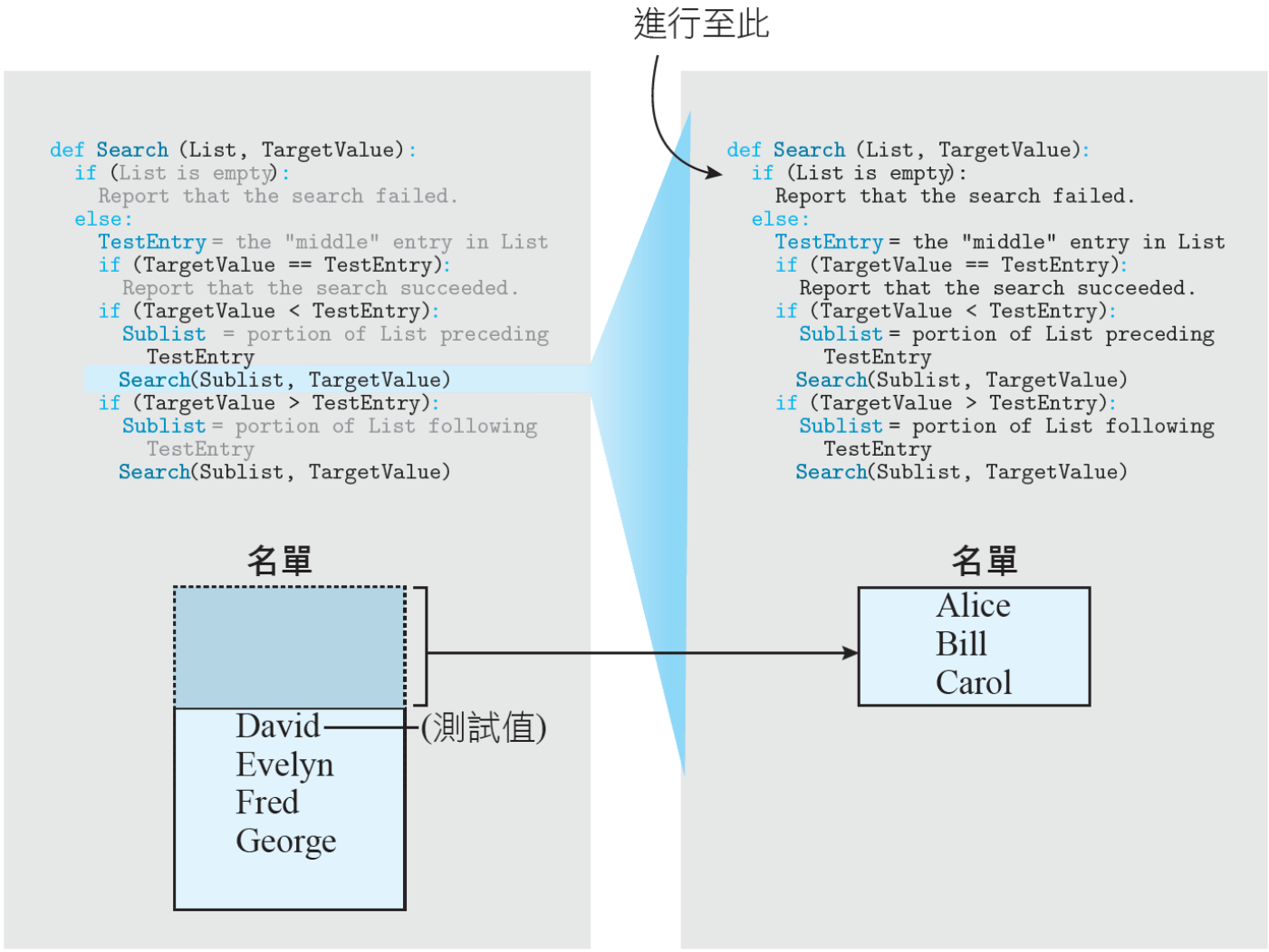
© 滄海圖書

圖5.14 二元搜尋法的虛擬碼

```
def Search(List, TargetValue):
  if (List is empty):
    Report that the search failed.
  else:
    TestEntry = the "middle" entry in List
    if (TargetValue == TestEntry):
      Report that the search succeeded.
    if (TargetValue < TestEntry):
      Sublist = portion of the List preceding TestEntry
      Search(Sublist, TargetValue)
    if (TargetValue > TestEntry):
      Sublist = the portion of List following TestEntry
      Search(Sublist, TargetValue)
```


© 滄海圖書

圖5.15 遞迴搜尋



© 滄海圖書

圖5.16 第二次遞迴搜尋的第一次截圖

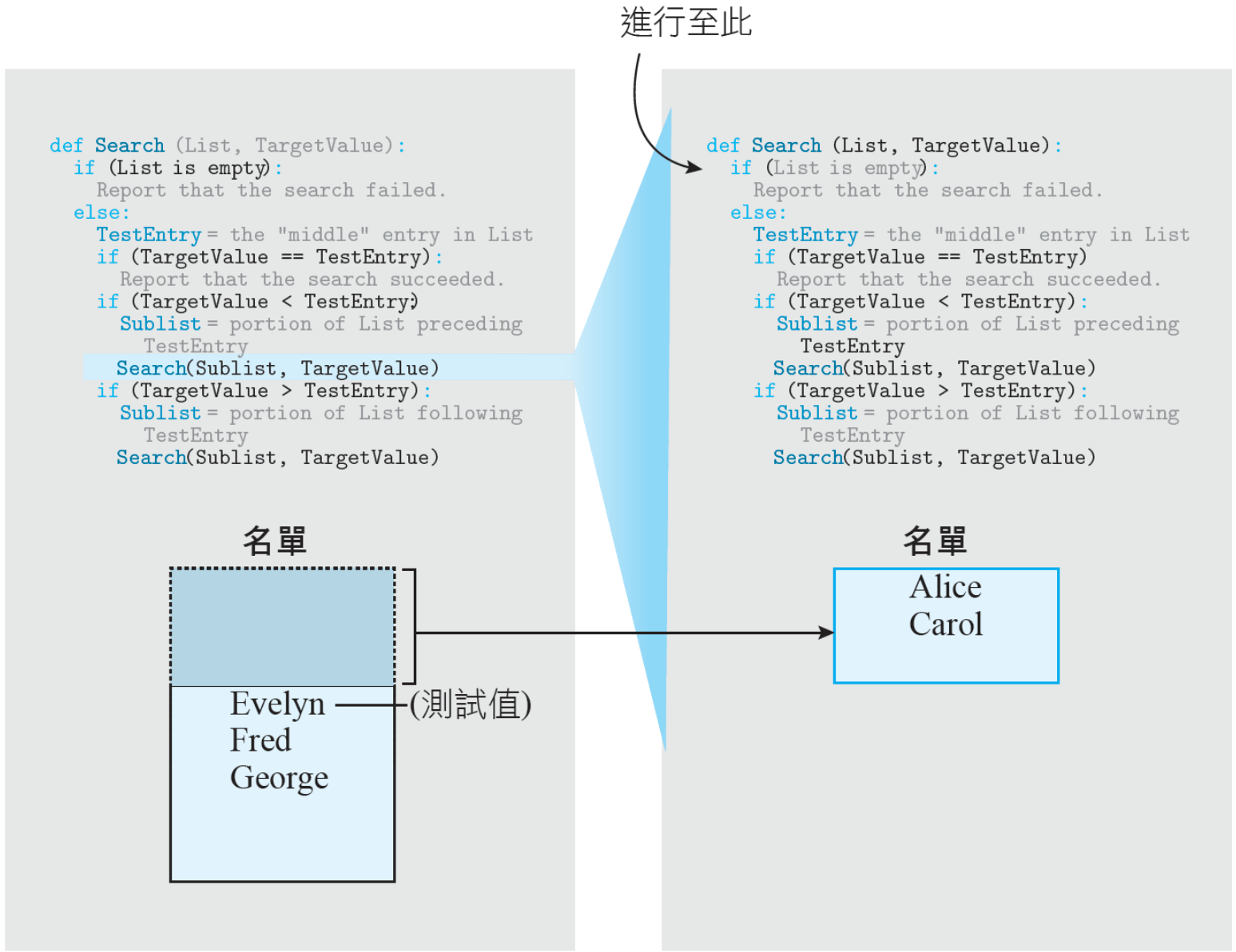
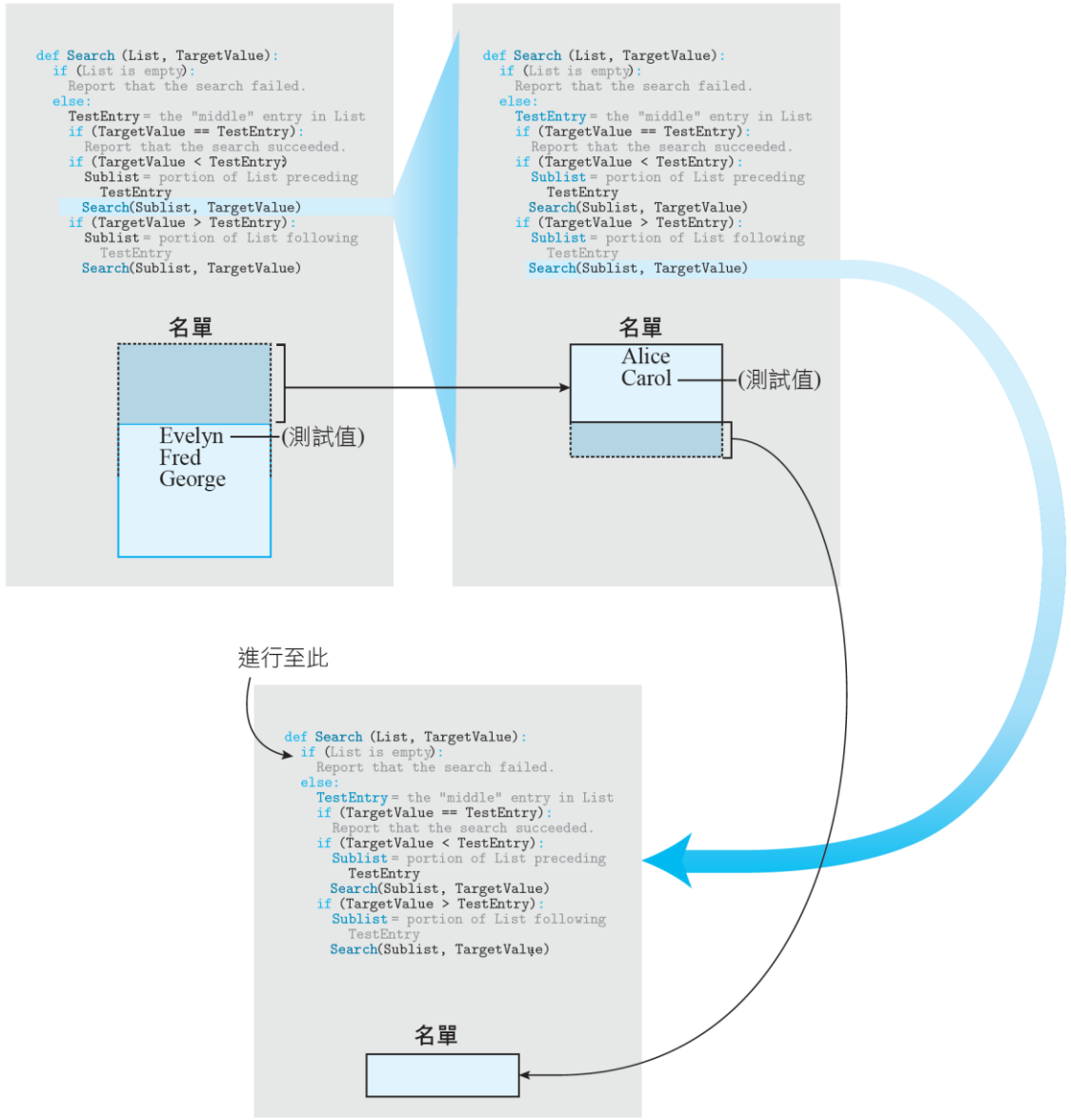


圖5.17 第二次遞迴搜尋的第二次截圖



© 滄海圖書

Example: Recursive Binary Search in Python

```python
def binary_search_recursive(list_lst, target_value, low =0, high = None):

    if high is None:
        high = len(list_lst) - 1

    if low > high:
        print(f"Cannot find the desired target value: {target_value}")
        return -1  # Search failed

    mid = (low + high) // 2
    test_entry = list_lst[mid]

    if target_value == test_entry:
        print(f"Search succeeded, found target value: {target_value}")
        return target_value, mid  # Search succeeded, return index

    elif target_value < test_entry:
        return binary_search_recursive(list_lst, target_value, low, mid - 1)

    else:
        return binary_search_recursive(list_lst, target_value, mid + 1, high)
```

```python
list = [1, 2, 3, 4, 5]
target_value, mid = binary_search_recursive(list, target_value: 1)
print(f"The target value:{target_value} is at the location: {mid} )")
```

```
def Search(List, TargetValue):
  if (List is empty):
    Report that the search failed.
  else:
    TestEntry = the "middle" entry in List
    if (TargetValue == TestEntry):
      Report that the search succeeded.
    if (TargetValue < TestEntry):
      Sublist = portion of the List preceding TestEntry
      Search(Sublist, TargetValue)
    if (TargetValue > TestEntry):
      Sublist = the portion of List following TestEntry
      Search(Sublist, TargetValue)
```

```
C:\Users\PTsai\PycharmProjects\pythonProje
Search succeeded, found target value: 1
The target value:1 is at the location: 0 )
```

© 滄海圖書

遞迴控制

- 需要初始化、變動和終止條件測試
- 如同望遠鏡般動態產生函數多個分身的錯覺
- 同一個時間，只有其中一個分身確實在執行，其他都在等待

© 滄海圖書

Exercise

- What sequence of numbers would be printed by the following recursive function if we started it with N assigned the value 1?
- What is the termination condition in the recursive function?

```
def Exercise (N):
    print(N)
    if (N < 3):
        Exercise(N + 1)
    print(N)
```

© 滄海圖書



In-Class Exercise

- What is the maximum number of entries that must be interrogated when applying the binary search to a list of 200 entries? What about a list of 100,000 entries?

© 滄海圖書



5.6 效率(Efficiency)與正確性(Correctness)

- 有效率與沒效率的演算法在實用性上有很大的差異
- 演算法的正確性取決於演算法正規的推論，而不是實作上的測試
- **結論**
  - **正確性（Correctness)**:由邏輯證明（理論、推導）保證。
  - **可靠性（Reliability):** 由實作測試（實際執行）檢查。

© 滄海圖書

演算法效率(Efficiency)

- 以需執行的指令個數來衡量
  - Algorithm analysis
    - Resources Usage(time(cpu), space(memory))
- 時間複雜度的**漸近**表示法: Ω、Θ、O (ignore the individual definition here)
  - Eg. 使用big theta表示法：
    - 範例：插入排序法屬於 $\Theta(n^2)$
- 以最佳(best Ω)、最壞(worst: O)和平均(average: Θ)情況(case)來分析
  - 一般以最壞情況來比較algorithms

© 滄海圖書

圖5.18 在最壞情況下使用插入排序法

每個軸心所需要的比對

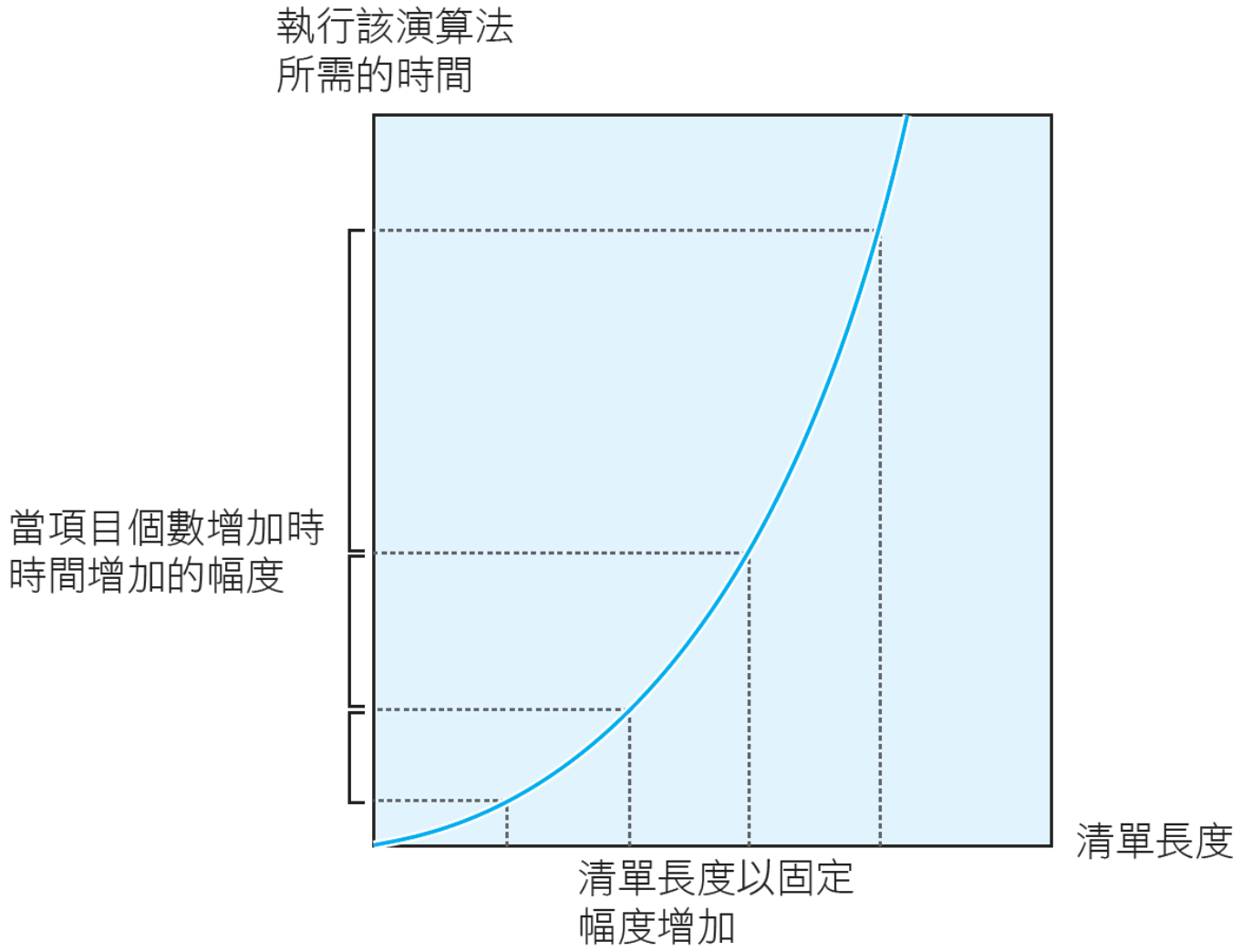
初始名單	1st 軸心	2nd 軸心	3rd 軸心	4th 軸心	已排序名單
Elaine	1 Elaine	3 David	6 Carol	10 Barbara	Alfred
David	David	2 Elaine	5 David	9 Carol	Barbara
Carol	Carol	Carol	4 Elaine	8 David	Carol
Barbara	Barbara	Barbara	Barbara	7 Elaine	David
Alfred	Alfred	Alfred	Alfred	Alfred	Elaine

情況	說明	時間複雜度
最佳情況	已排序的資料，只需比較一次	(O(n))
平均情況	資料隨機	(O(n^2))
最壞情況	**資料反序**，每次都需移動全部元素	**(O(n^2))**

$$1+2+3+\cdots+(n-1)=\frac{n(n-1)}{2}$$

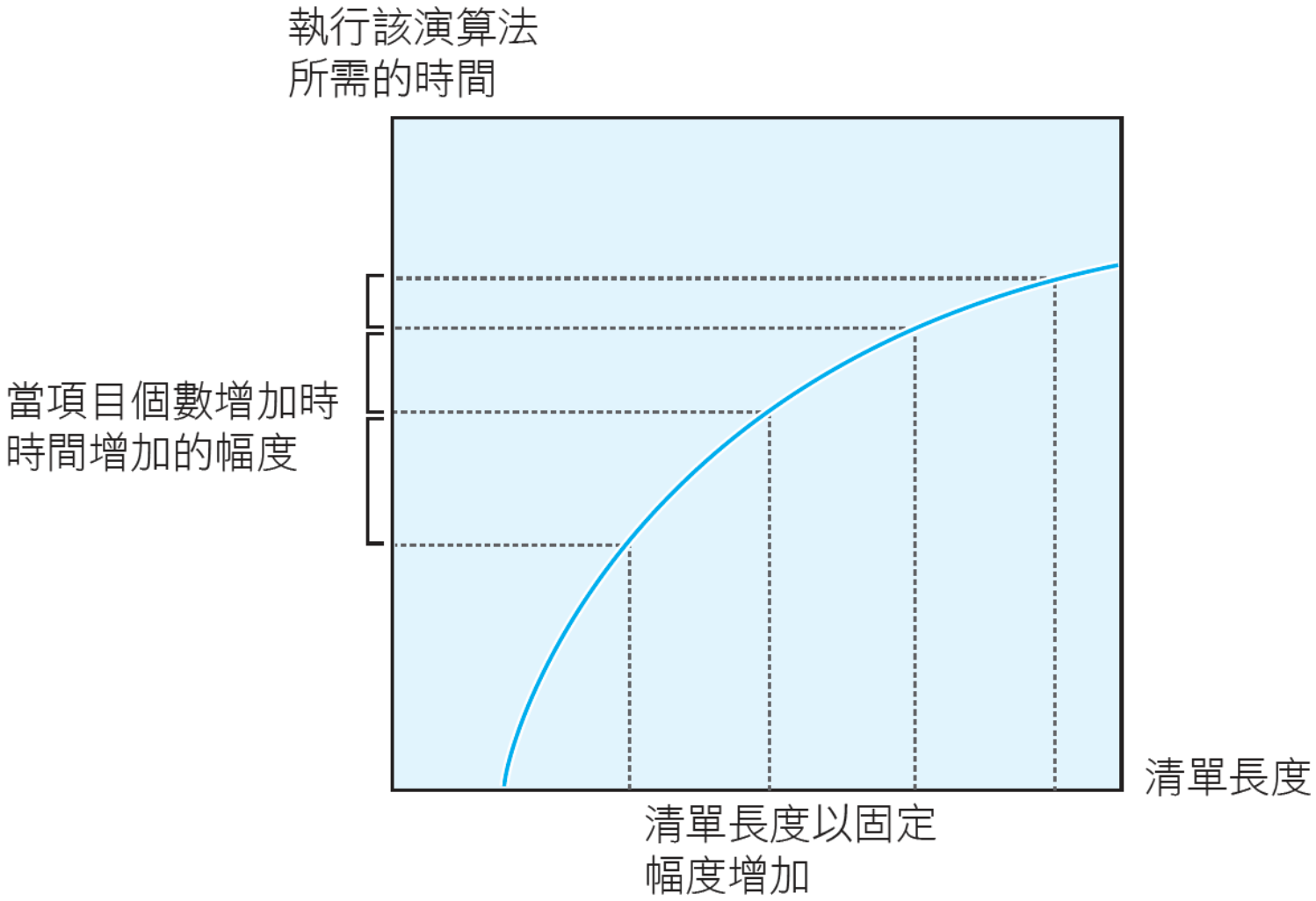
© 滄海圖書

圖5.19 插入排序法的最壞情況分析的圖形



© 滄海圖書

圖5.20 二元搜尋法的最壞情況分析的圖形



© 滄海圖書

Time Complexity Comparison

Sequential			Insertion Sort			Binary		
Best	average	worse	Best	average	worse	Best	average	worse
1	$n/2$	n	$n-1$	$\frac{1}{4}(n^2-n)$	$\frac{1}{2}(n^2-n)$	1	$\log_2 n$	$\log_2 n$

© 滄海圖書



Space Complexity

Algorithm	Auxiliary Space	Type	Notes
Sequential Search	O(1)	In-place	Uses only a few variables for indexing and comparison
Insertion Sort	O(1)	In-place	Just a few variables (key, indices); no new arrays created
Binary Search (iterative)	O(1)	In-place	Only low, high, mid variables used
Binary Search (recursive)	O(log n)	Recursive	Each recursive call adds to call stack (depth = log n)

© 滄海圖書



Example

- Insertion Sort
  - Supposed:
    - n = 10
    - Best

$$10-1=9 \textbf{ comparisons}$$

    - Average

$$\frac{1}{4}(10^2-10)=22.5 \textbf{ comparisons}$$

    - Worse

$$\frac{1}{2}(10^2-10)=45 \textbf{ comparisons}$$

© 滄海圖書



大O估計

- **定理**：令$f(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$，其中$a_0, a_1, \ldots, a_{n-1}, a_n$皆為實數。則，$f(x)$為$O(x^n)$。

~ (Rosen, 2013)

© 滄海圖書



Time Complexity

Algorithm	Best Case	Average Case	Worst Case	Description
Sequential Search	O(1)	O(n)	O(n)	Best case: target found at first element; worst case: not found or last element
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	Best: already sorted; worst: reverse order
Binary Search	O(1)	O(log n)	O(log n)	Always divides search range by 2 each step (requires sorted list)

© 滄海圖書

Exercises

1. 假設執行時間$T(n) = 3n^3 + 2n^2 + 5n$，求時間複雜度為何?

2. 考慮下列$x = x + 1$的執行數。

```
1  .
2  .
3  x=x=1
4  .
5  .
```

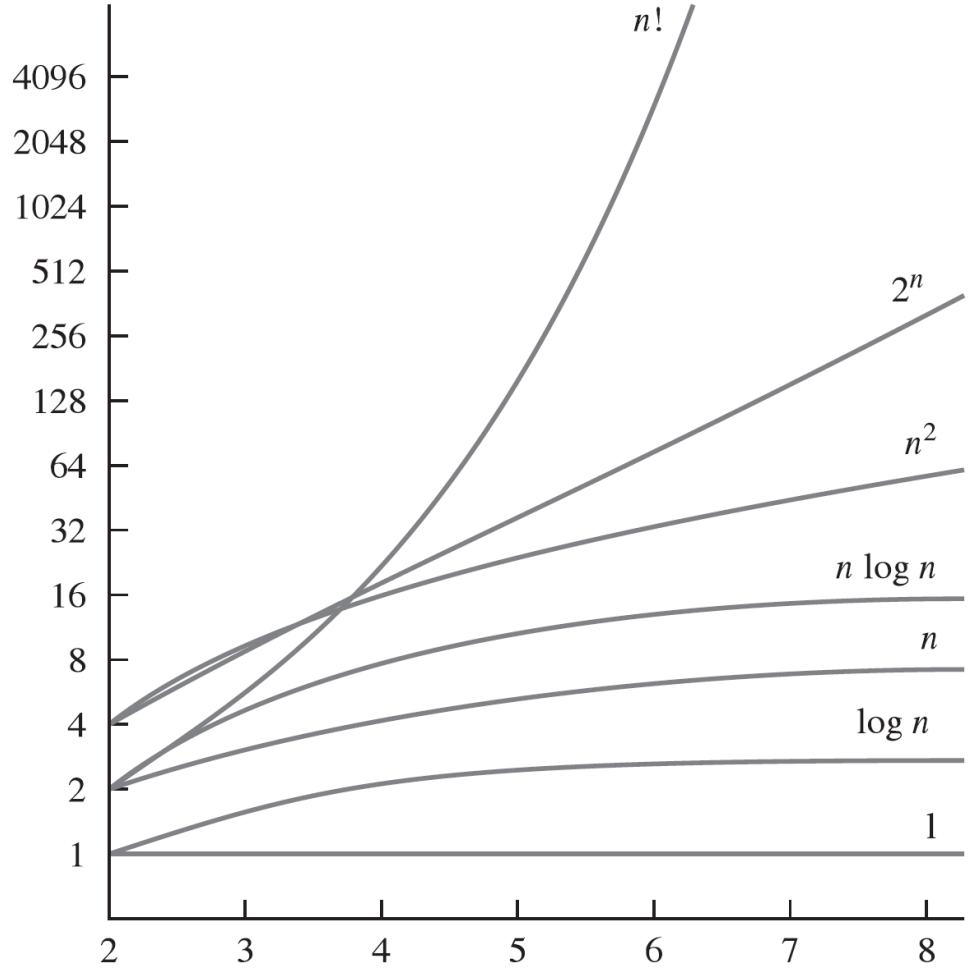
```
x = 0
n = 4
for i in range(1, n+1):
    x = x+1
print(x)
```

```
x = 0
n = 4
m = 4
for i in range(1, n+1):
    for j in range(1, m+1):
        x = x+1
print(x)
```

© 滄海圖書

To Wrap up:演算法複雜度((Complexity))的常用術語(Terminology) ~(Rosen, 2013)

複雜度	術語
O(1)	常數(constant)複雜度
O(log n)	對數(logarithmic)複雜度
O(n)	線性(linear)複雜度
O(n log n)	n log n複雜度
O(n^b)	多項式(polynomial)複雜度
O(b^n)，當$b > 1$	指數(exponential)複雜度
O($n!$)	階乘(factorial)複雜度



© 滄海圖書

演算法所使用的電腦運算時間 ~(Rosen, 2013)

問題規模	使用之位元運算					
n	$\log n$	n	$n \log n$	n^2	2^n	$n!$
10	3×10^{-11} s	10^{-10} s	3×10^{-10} s	10^{-9} s	10^{-8} s	3×10^{-7} s
10^2	7×10^{-11} s	10^{-9} s	7×10^{-9} s	10^{-7} s	4×10^{11} yr	*
10^3	1.0×10^{-10} s	10^{-8} s	1×10^{-7} s	10^{-5} s	*	*
10^4	1.3×10^{-10} s	10^{-7} s	1×10^{-6} s	10^{-3} s	*	*
10^5	1.7×10^{-10} s	10^{-6} s	2×10^{-5} s	0.1 s	*	*
10^6	2×10^{-10} s	10^{-5} s	2×10^{-4} s	0.17 min	*	*

*: 所需時間超過10^{100}年時;

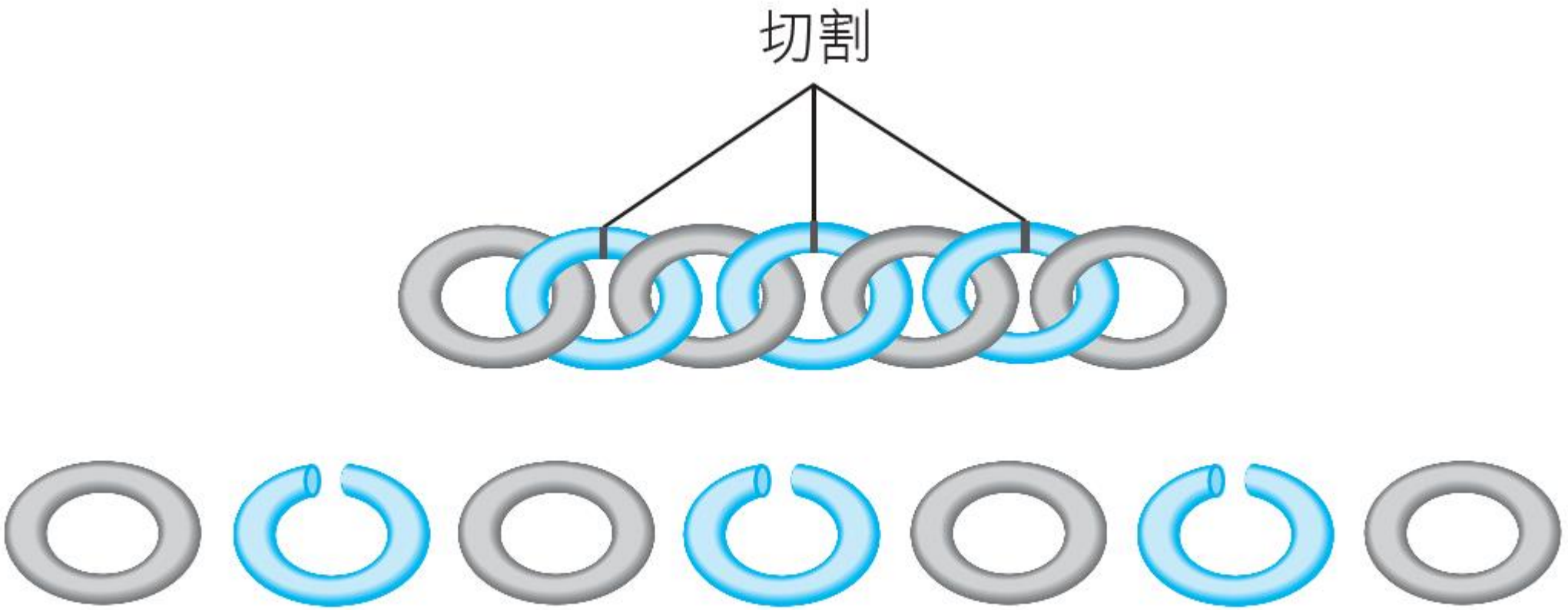
© 滄海圖書

Example: 金環切割問題(Chain Separating Problem)

- 有個旅者帶著七個相連的金環鍊子
- 他必須在一個孤立的旅館中住七個晚上
- 每晚的房租是一個金環
- 若旅者只想要每天早上付給旅館一個金環，而不想一次預付好幾個金環，試問至少需切割多少個金環？

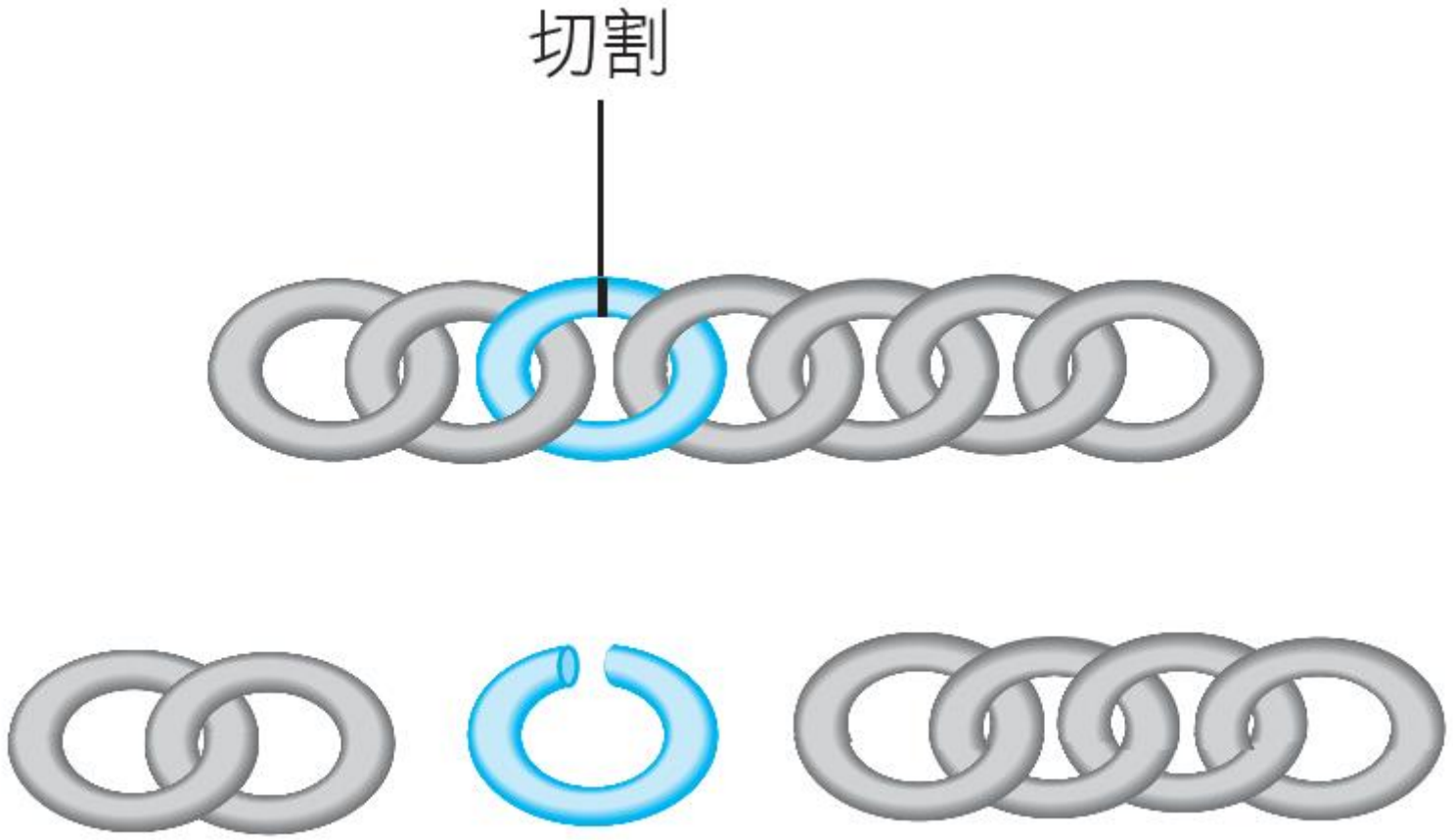
© 滄海圖書

圖5.21 切割三個金環就能分開所有的金環



© 滄海圖書

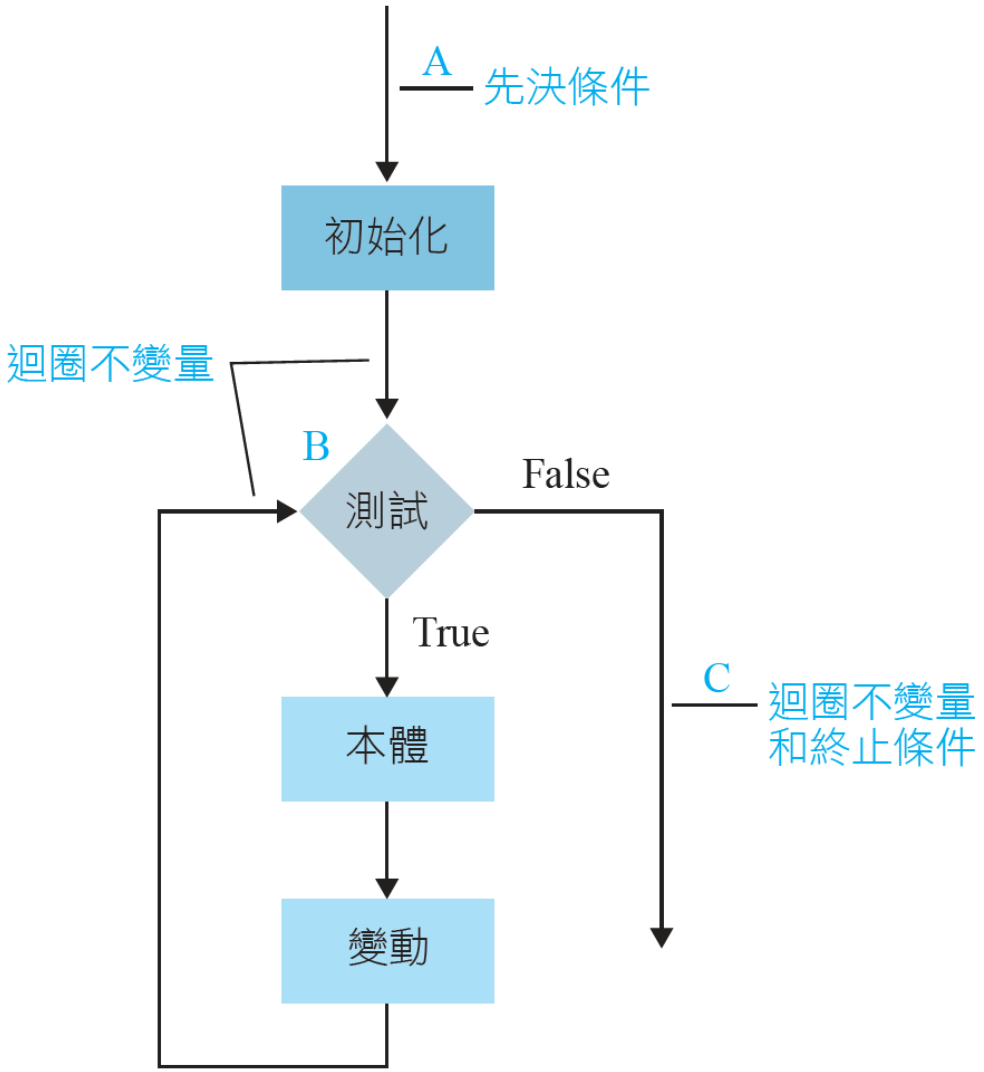
圖5.22 切割一個金環來解該問題



© 滄海圖書

軟體驗證(Software Verification)

- 程式的正確性證明 (使用正規邏輯)
  - 判定(Assertion)
    - 先決條件(Precondition)
      - initial assertion
    - 迴圈不變量(Loop Invariant)
      - e.g. 每次進行終止條件測試後，名單中的第1項到第N-1
    - 後置條件(Postconditions)
      - final assertion
- 更常使用測試(Testing)來驗證軟體
  - 測試只能證明程式在測試案例中是正確的



軟體驗證(Cont.)

- how to know that the algorithm we designed to solve the problem is correct in terms of all situations?

p->initial assertion
S->a program, or program segment
q->final assertion

Rules of Inference (composition rule):
p{S1}q
q{S2}r

---

so p{S1; S2}r ->true(correct)

© 滄海圖書



Definition of Loop invariant

- 非正式地說，**迴圈不變量**是關於某個迴圈的一個陳述，且在**每次迭代前與後**（包含第一次與最後一次）都為真。
- **定義：**

  形式上，迴圈不變量必須滿足下列性質：

  - **初始化（Initialization）：** 在迴圈第一次迭代之前，迴圈不變量為真。
  - **維持（Maintenance）：** 如果在某次迭代開始時迴圈不變量為真，則在下一次迭代開始時它仍為真。
  - **終止（Termination）：** 當迴圈結束時，迴圈不變量提供一個有用的性質，可用來證明演算法是正確的。

© 滄海圖書



Example: Loop Invariant

- Loop Invariant?

```
def sum_of_natural_numbers(n):
    sum = 0
    i = 1
    assert( n > 0)
    while i <= n:
        sum = sum + i
        i = i + 1
    return sum

sum = sum_of_natural_numbers(2)
assert( sum == 3)
print (sum)
```

```
3
```

Initialization

Maintenance

Termination:

sum contains the sum of the first i natural numbers

© 滄海圖書



總結

- [redacted]是一種在每次迴圈迭代之前和之後都為真的條件或斷言。這是計算機科學中的一個基本概念，用來推理迴圈的[redacted]。

- 當滿足這三個條件（[redacted]、[redacted]、[redacted]）時，這就保證了迴圈將正確執行。如果迴圈不變量成立，這意味著迴圈正在朝著終止條件進展，並且最終將以所需的結果終止。

- 使用[redacted]是程式正式驗證和算法分析中的一項重要技術。它有助於理解、證明和調試(透過除錯)迴圈的正確性。

© 滄海圖書